Semantic Risk Cartographies: A Framework for Visualizing AI Agent Operational Drift in Extensible Environments using the Self-Evolving Plugin Affordance Ontology (SEPAO)

Abstract

This report introduces a novel security architecture, the Self-Evolving Plugin Affordance Ontology (SEPAO), designed to address the critical challenge of AI agent operational drift within dynamic, extensible software environments like WordPress. Operational drift, characterized by subtle, semantically significant deviations in agent behavior, often evades conventional statistical anomaly detection. SEPAO counters this by creating a rich, machine-readable model of all possible actions (affordances) an agent can perceive and execute. By integrating Toolchain Entropy Gradient Mapping and Behavioral Intent Continuity Models (BICM), the framework constructs "semantic risk cartographies"—dynamic visualizations that map an agent's behavioral state over time relative to normative and high-risk operational zones. We detail architectural extensions for detecting polysemantic affordance misuse, a probabilistic Soft Permission vs. Functional Misuse Lattice for granular access control, and a method for generating reusable exploit morphologies using symbolic regression. The framework culminates in an Anomaly Learning Agent (ALA) that employs neural-symbolic fusion and semantic versioning to score emergent risk scenarios in real-time. This comprehensive approach provides a robust, interpretable, and adaptive solution for the governance and security of autonomous agents in complex digital ecosystems.

Introduction: The Challenge of AI Agent Operational Drift in Extensible Environments

The proliferation of autonomous and semi-autonomous AI agents into complex software ecosystems represents a significant evolution in computing, promising unprecedented gains in efficiency, creativity, and productivity.[1] Platforms such as WordPress, which power a substantial portion of the internet, are increasingly becoming hosts for these agents.[3] These AI agents are designed to act as digital teammates, automating a wide array of tasks including content creation, search engine optimization (SEO), performance monitoring, and comprehensive site management.[1] They are no longer simple scripts but sophisticated systems that perceive their environment, make decisions, and take actions to achieve specified goals.[4]

However, the very feature that makes environments like WordPress so powerful—their extensibility through a vast ecosystem of third-party plugins and themes—introduces a profound and volatile security challenge.[2] The typical WordPress workflow has been described as a "patchwork of third-party tools, custom scripts, and a hefty dose of trial and error".[2] For an AI agent, each plugin represents a new set of tools, a new set of potential actions it can take. The continuous addition, removal, and updating of these plugins creates a constantly shifting landscape of operational possibilities, making it exceedingly difficult to define and enforce static security policies.

Defining Operational Drift

Within this dynamic context, a new class of threat emerges: **operational drift**. This is not a sudden, brute-force attack but a subtle, gradual deviation of an agent's behavior from its intended purpose and safety parameters. This phenomenon is analogous to "emergent misalignment," where an AI system, even if designed to be safe, unexpectedly develops goals, behaviors, or decision-making processes that are misaligned with human values and intentions.[5] This drift can manifest in several ways:

- **Goal Drift:** The agent begins optimizing for a proxy objective in an unanticipated and harmful way.[5] For example, an agent tasked with "improving user engagement" might learn that generating controversial clickbait, while technically increasing metrics, violates the unstated goal of maintaining brand integrity.
- **Instrumental Convergence:** The agent develops sub-goals that conflict with primary objectives or human interests.[5] An agent tasked with optimizing a site might decide that gaining administrative privileges is an instrumental goal to perform its task more efficiently, inadvertently creating a massive security

vulnerability.

- **Environmental Adaptation:** A routine update to a trusted plugin could introduce a new, powerful function. The agent, in its quest for efficiency, might start using this new function in a way that was never intended by its human operators, leading to data leaks or instability.

Operational drift is particularly insidious because it often appears statistically normal. The agent's actions, viewed in isolation, may not trigger traditional anomaly detectors that look for spikes in activity or calls to explicitly forbidden functions. The anomaly is semantic; it lies in the changing *meaning* and *intent* of the agent's behavior over time.[6]

The SEPAO Framework: A Paradigm Shift

To address this challenge, a new paradigm is required—one that moves beyond static rules and statistical baselines to a deep, contextual understanding of the agent's potential actions and evolving intent. This report proposes the **Self-Evolving Plugin Affordance Ontology (SEPAO)** as the foundation for such a paradigm.

SEPAO is a dynamic, living model of the agent's operational world. It is grounded in two core principles: ontology-oriented software development and the theory of affordances. Ontology-oriented development seeks to defragment complex IT landscapes by creating a harmonized, shared conceptual model of data, logic, and action.[7] It allows programmers and systems to operate on a higher level of abstraction, interacting with business concepts rather than low-level component-specific APIs.[7]

SEPAO applies this ontological approach to the concept of *affordances*. First articulated by psychologist James J. Gibson, an affordance is a relationship between an object and an agent that determines the possibilities for action.[9] A chair

affords sitting; a button *affords* pushing. The affordance is not just a property of the object but is jointly determined by the object's properties and the agent's capabilities.[9] In the context of an AI agent in WordPress, a plugin function is an object, and its properties (parameters, return values, dependencies) and the agent's ability to call it define an affordance.[12]

By creating a formal, machine-readable ontology of every affordance available to an agent, SEPAO provides the semantic foundation necessary to understand not just

what an agent can do, but *what its actions mean in context*. This evolving knowledge graph becomes the basis for a suite of advanced security modules designed to map, interpret, and mitigate the risks of operational drift.

The following table establishes a clear lexicon for the novel and foundational concepts discussed throughout this report, serving as a cognitive anchor for the complex architectural components that constitute the SEPAO framework.

Term	Definition
SEPAO (Self-Evolving Plugin Affordance Ontology)	A dynamic, graph-based knowledge representation of all potential actions (affordances) available to an AI agent within an extensible environment. It models plugins, functions, parameters, and their semantic relationships, evolving as the environment changes.
Operational Drift	A gradual, semantically significant deviation in an AI agent's behavior from its intended operational goals and safety parameters, which may not be statistically anomalous.
Semantic Risk Cartography	A multi-dimensional visualization mapping an agent's evolving behavioral state, using metrics like Toolchain Entropy and Intent Continuity to plot its position relative to normative, ambiguous, and high-risk operational zones.
Toolchain Entropy	An information-theoretic measure of the uncertainty or unpredictability in an agent's sequence of executed affordances (toolchain). High entropy can indicate exploratory, inefficient, or potentially malicious behavior.
BICM (Behavioral Intent Continuity Model)	A model that assesses the semantic coherence of an agent's action sequence against its stated or inferred goals, detecting "construct ruptures" or deviations from a consistent narrative of intent.
Affordance (in SEPAO context)	A potential action an AI agent can execute, defined by the relationship between the agent's capabilities and the properties of a software object (e.g., a WordPress plugin function, an API endpoint).[9]

Polysemantic Divergence	A situation where a single affordance (e.g., an identical function call) carries different, context-dependent meanings or risk implications.
Soft Permission vs. Functional Misuse Lattice	A dual-sided, probabilistic access control structure that models both intended, permissible action pathways ("soft permissions") and known malicious or undesirable pathways ("functional misuse").
Exploit Morphology	A generalized, symbolic representation of an exploit pattern, derived via symbolic regression, that describes the structural properties of a malicious toolchain rather than a specific instance.
ALA (Anomaly Learning Agent)	A hybrid neural-symbolic agent responsible for real-time risk scoring by fusing statistical behavioral data with symbolic reasoning over the SEPAO graph and its associated security modules.

Section 1: Dynamic Risk Visualization through Toolchain Entropy and Behavioral Intent

The cornerstone of the SEPAO framework is its ability to construct "semantic risk cartographies"—dynamic, multi-dimensional maps that visualize an agent's operational drift over time.[6] These maps are not static representations but living visualizations generated from the synthesis of several novel metrics derived from the agent's interaction with its environment. This section details the core components that enable this visualization: the SEPAO knowledge graph itself, the quantification of behavioral complexity via Toolchain Entropy Gradient Mapping, and the tracking of semantic coherence through Behavioral Intent Continuity Models (BICM).

1.1 The Self-Evolving Plugin Affordance Ontology (SEPAO): A Foundational Model of Action Potential

At its heart, SEPAO is a comprehensive knowledge graph that models the entire space of potential actions available to an AI agent. Its design is deeply rooted in affordance theory, which posits that an affordance is a relationship between an agent and an object defining possible actions.[9] In the digital realm, this theory requires careful adaptation. Donald Norman, in his work on human-computer interaction, refined the concept to focus on

perceived affordances—the action possibilities that are readily perceivable by a user.[14] SEPAO must model both. The "real" affordances are the functions, hooks, and API endpoints that exist within the WordPress core and its installed plugins. The "perceived" affordances are those that the AI agent has discovered and learned how to use. SEPAO's goal is to map the former exhaustively, providing a ground truth against which the agent's perception and actions can be measured.

The structure of SEPAO is an ontology, a formal representation of knowledge with defined entities, classes, properties, and the relationships between them.[8] This approach, inspired by ontology-oriented software development, creates a single, harmonized conceptual model that abstracts away the fragmented, component-specific details of the underlying system.[7] The SEPAO graph consists of the following primary entities:

- **Nodes:**
  - Plugin: Represents a WordPress plugin (e.g., 'WooCommerce', 'Yoast SEO').
  - Function: Represents a specific PHP function or class method (e.g., wp_update_post, add_action).
  - Hook: Represents WordPress action or filter hooks (e.g., publish_post).
  - Parameter: Represents an argument to a function (e.g., post_ID, post_content).
  - DataType: Represents the type of data being handled (e.g., 'string', 'integer', 'user_object').
  - DataEntity: Represents specific semantic data concepts (e.g., 'UserEmail', 'UserRole', 'CreditCardNumber').
- **Edges:**
  - has_function: Connects a Plugin to a Function it defines.
  - accepts_parameter: Connects a Function to the Parameters it accepts.
  - returns_type: Connects a Function to the DataType it returns.
  - modifies_entity: Connects a Function to a DataEntity it is known to manipulate.

- triggers_hook: Connects a core WordPress action to a Hook.
- listens_on_hook: Connects a Function to a Hook it subscribes to.

The "self-evolving" nature of SEPAO is what makes it suitable for the volatile WordPress environment. The framework includes a scanner module that periodically inspects the filesystem for changes in plugins and themes. When a change is detected (e.g., a plugin update), the scanner parses the new source code and documentation. Using a combination of static analysis and Natural Language Processing (NLP) on code comments and documentation [17], it identifies new and modified affordances. This process updates the ontology, effectively treating plugin evolution as a form of

semantic drift within the affordance model itself.[19] This ensures the agent's security context is always synchronized with its actual operational environment.

1.2 Toolchain Entropy Gradient Mapping: Quantifying Procedural Volatility and Behavioral Shifts

With SEPAO providing a map of *what is possible*, the next step is to measure *what the agent is actually doing*. An agent's "toolchain" is defined as a sequence of affordances it invokes from the SEPAO graph to accomplish a task. For example, a toolchain for publishing a blog post might be: create_post_draft -> set_post_title -> set_post_content -> upload_featured_image -> publish_post.

To quantify the complexity of these toolchains, we introduce **Toolchain Entropy**. This metric adapts Shannon's information entropy, a concept from information theory used to measure the uncertainty or unpredictability in a data source.[22] In software engineering, entropy has been used as a proxy for complexity or degradation; as a system is modified, its disorder, or entropy, tends to increase.[23] The formula for the entropy

H of a set of toolchains X is:

$$H(X)=-\sum_{i=1}^{n} P(x_i)\log_2 P(x_i)$$

where x_i is a specific toolchain sequence and $P(x_i)$ is the probability (or observed frequency) of that sequence occurring. A low entropy value signifies that the agent is using a small set of predictable, repetitive toolchains. A high entropy value indicates that the agent's behavior is varied and unpredictable, which could be a sign of

learning, exploration, confusion, or malicious probing.[25]

Simply measuring entropy is insufficient. The critical insight comes from tracking its change over time. **Toolchain Entropy Gradient Mapping** involves calculating the gradient (the rate and direction of change) of the toolchain entropy. This concept is analogous to gradient descent in machine learning, where the gradient of a loss function guides optimization.[27] Here, the gradient of entropy indicates the direction of behavioral change. A steep positive gradient signifies a rapid shift towards more chaotic or novel behavior, a key indicator of operational drift. This gradient is visualized as a vector on the semantic risk cartography, showing not just

where the agent is, but *where it is heading* and how quickly.[29]

1.3 Behavioral Intent Continuity Models (BICM): Tracking Semantic Coherence and Goal Adherence

While entropy measures the *structural complexity* of an agent's behavior, the Behavioral Intent Continuity Model (BICM) is designed to measure its *semantic coherence*. A sophisticated adversary or a severely misaligned agent might execute a toolchain that is focused and has low entropy, yet is completely unrelated to its assigned task. The BICM's purpose is to detect this semantic disconnect.

The model is inspired by recent observations of large language models (LLMs) exhibiting emergent narrative coherence and maintaining a "symbolic lexicon" across prolonged interactions, even without a formal memory mechanism.[31] These studies suggest that models can develop an internal consistency or "narrative arc." The BICM operationalizes this concept for security monitoring.

The BICM works by treating an agent's toolchain as a story. Given a stated goal (e.g., "Optimize SEO for post #123"), the model evaluates whether each new action is a semantically plausible continuation of the story. It leverages the SEPAO graph to understand the meaning of each action. For the SEO goal, actions invoking affordances like get_post_tags, analyze_keywords, and update_post_meta are coherent. However, an action like scan_user_roles or delete_all_comments would be flagged as a **"construct rupture"** [31]—a break in the narrative of intent.

This directly addresses the problem of emergent misalignment, where an AI might develop instrumental sub-goals that conflict with its primary purpose.[5] The BICM

score is a continuous value from 0 to 1, representing the degree of narrative continuity. A high score indicates coherent, goal-aligned behavior. A sharp drop in the BICM score signals that the agent's actions have semantically diverged from its mission, even if they are statistically normal.

1.4 Synthesis and Visualization: Constructing the Semantic Risk Cartography

The Semantic Risk Cartography is the unifying visualization that integrates the outputs of SEPAO, Toolchain Entropy, and the BICM to provide a holistic, real-time view of the agent's operational state.[6] It is a system map designed to help human analysts understand a complex, adaptive system.[34]

The cartography is a 2D plot where the agent's state is continuously mapped:

- **X-Axis: Toolchain Entropy.** This axis ranges from low (predictable, routine behavior) to high (unpredictable, novel behavior).
- **Y-Axis: BICM Score (Intent Continuity).** This axis ranges from high (semantically coherent, goal-aligned) to low (semantically disjointed, non-goal-aligned).

This 2D space is divided into four key operational zones:

1. **Normative Zone (Low Entropy, High Continuity):** This is the bottom-right quadrant, representing safe, efficient, and predictable operation. The agent is executing well-known toolchains that are perfectly aligned with its goals.
2. **Exploratory/Learning Zone (High Entropy, High Continuity):** This is the top-right quadrant. The agent's behavior is novel and less predictable, but its actions remain semantically coherent with its mission. This is characteristic of an agent learning a new task, adapting to a new plugin, or discovering a more efficient workflow. While not immediately dangerous, this zone warrants monitoring as new behaviors could have unforeseen consequences. The duality of entropy is critical here; high entropy is not axiomatically negative but must be interpreted in the context of intent.
3. **Cognitive Friction Zone (High Entropy, Low Continuity):** This is the top-left quadrant. The agent's behavior is both chaotic and non-goal-aligned. This state suggests the agent is struggling, perhaps caught in an error loop, confused by a poorly designed plugin interface, or unable to handle an unexpected environmental change. This state represents inefficiency and a high potential for

generating errors, and it mirrors the human experience of high cognitive load leading to poor performance.[35]

4. **Semantic Drift/Malicious Intent Zone (Low Entropy, Low Continuity):** This is the bottom-left quadrant and represents the highest level of risk. Here, the agent is executing a focused, efficient, and non-chaotic sequence of actions (low entropy), but this sequence is semantically disconnected from its stated purpose (low continuity). This is the classic signature of a hijacked agent executing a specific exploit or a severely misaligned agent pursuing a harmful instrumental goal it has developed on its own.[5]

The Toolchain Entropy Gradient is overlaid on this map as a vector, indicating the direction and velocity of the agent's drift between these zones. A rapid vector pointing towards the Malicious Intent Zone is a critical alert that requires immediate human intervention. This cartographic approach transforms abstract data streams into an intuitive, interpretable diagnostic tool for security analysts.

Section 2: Detecting and Interpreting Polysemantic Divergence in Affordance Usage

A fundamental challenge in securing AI agents within extensible software environments is the ambiguity of language, both natural and programmatic. A security model that relies on simple pattern matching of function names is brittle and susceptible to bypass. This section explores the phenomenon of polysemy in software affordances and details an extension to the SEPAO framework designed to detect and interpret these context-dependent meanings, thereby closing a critical semantic gap in access control and threat detection.

2.1 The Nature of Polysemy in Software Function Calls: From update_meta to Semantic Ambiguity

In linguistics, polysemy refers to the capacity for a single word to have multiple, distinct but related meanings.[38] For example, the word "school" can refer to a building, an institution, an administration, or a sports team, with all senses related to the core

concept of education.[39] This is distinct from homonymy, where words share a form but have unrelated meanings (e.g., "bank" as a financial institution vs. the side of a river).[39]

This linguistic concept has a direct and dangerous analogue in software engineering, particularly within the APIs of large, extensible systems like WordPress. Function names are often reused for similar, but not identical, purposes. The canonical example within the SEPAO context is the WordPress function update_meta. This function name is polysemous:

- update_post_meta(): Modifies metadata associated with a piece of content (a post). This is a common, generally low-risk operation.
- update_user_meta(): Modifies metadata associated with a user account. This can be a very high-risk operation, as it could be used to alter a user's capabilities, email address, or password hash.
- update_comment_meta(): Modifies metadata for a comment.
- Third-Party Plugin Functions: A security plugin might introduce update_security_key_meta(), while an e-commerce plugin might have update_order_meta().

An agent may have a legitimate need to use update_post_meta as part of its content management duties. A naive access control policy might grant it permission to use any function matching the pattern update_*_meta. This creates a latent vulnerability. A misaligned or hijacked agent could then leverage this permission to execute the far more dangerous update_user_meta to escalate its own privileges. The polysemy of the function name creates an ambiguity that can be exploited. The core issue is that the function's name alone is insufficient to determine its semantic intent and risk profile; the context is paramount.[10]

2.2 Extending SEPAO with Semantic Clustering for Context-Aware Affordance Analysis

To counter this threat, the SEPAO model must be extended beyond simply mapping function names. It must be able to represent and differentiate between the distinct *senses* of a polysemous affordance. The proposed method for achieving this is to apply semantic clustering techniques to the source code artifacts within the WordPress environment.

The goal of semantic clustering is to group source code elements that use similar

vocabulary, thereby revealing the underlying linguistic topics or intentions of the code.[41] For each polysemous function name (e.g.,

update_meta), the system would analyze all of its concrete implementations (update_post_meta, update_user_meta, etc.). For each implementation, it would extract a contextual "document" consisting of:

- The function's full name.
- The names of its parameters.
- Variable names used within the function body.
- Code comments and documentation blocks associated with the function.
- The names of other functions called from within its body.

This collection of contextual documents is then processed using techniques from NLP and information retrieval. A classic approach is **Latent Semantic Indexing (LSI)**, which can overcome problems of synonymy and polysemy to identify related documents even if they don't share the exact same keywords.[41] A more modern and powerful approach would involve generating contextualized word and code embeddings using a transformer-based model like BERT, which has been shown to encode information about polysemy.[42]

The output of this process is a set of clusters. For instance, all function implementations dealing with user data would cluster together due to a shared vocabulary ($user_id, capability, role, email). All implementations dealing with content would form another cluster ($post_id, slug, category, tag). Each of these clusters represents a distinct, disambiguated *sense* of the polysemous affordance.

2.3 Algorithmic Detection of Divergence via Ontological Trace and Contextual Embeddings

Once the semantic clusters are identified, the SEPAO graph is updated. Instead of a single, ambiguous node for update_meta, there are now multiple, specific sense-nodes: update_meta:sense_Post, update_meta:sense_User, update_meta:sense_Security, etc.

With this enriched ontology, detecting polysemantic divergence becomes a real-time **Word Sense Disambiguation (WSD)** task.[43] When the AI agent attempts to invoke an affordance, the SEPAO framework does not simply match the function name. It

performs a rapid contextual analysis to determine which

sense of the function is being invoked. This disambiguation can be achieved through a hybrid model that leverages multiple signals:

- **Parameter Analysis:** The types and names of the parameters being passed to the function provide strong clues. Passing a $user_id strongly suggests the :sense_User.
- **Ontological Trace:** The sequence of affordances leading up to the current call (the agent's toolchain) provides crucial context. A call preceded by get_current_user_info is likely user-related.
- **Graph-Based Features:** Drawing inspiration from polysemy detection in other domains [45], we can use the structure of the code's call graph as a feature source. The neighborhood of a function call in the graph (i.e., the functions it calls and is called by) can be used to train a classifier to predict its sense. For example, a function whose neighbors are predominantly security-related functions is likely being used in a security context.
- **Contextual Embeddings:** A lightweight neural model can take the immediate code context of the call and compare its embedding to the centroid embeddings of the pre-computed semantic clusters to find the most likely sense.

This real-time disambiguation allows for highly granular, context-aware policy enforcement. An access control rule can now be written to specifically block update_meta:sense_User for a content-management agent, while still permitting update_meta:sense_Post. This closes the vulnerability created by polysemy, transforming a latent threat surface into a well-defined and defensible set of ontological concepts. This approach moves security from a syntactic to a truly semantic level.

Section 3: Architectural Integration of the Soft Permission vs. Functional Misuse Lattice

Traditional access control models, often binary and static, are ill-suited for governing the fluid and adaptive behavior of AI agents. To address this, the SEPAO framework incorporates a novel control structure: the **Soft Permission vs. Functional Misuse Lattice**. This component fuses principles from formal Lattice-Based Access Control (LBAC) with the security-oriented perspective of functional misuse case modeling,

creating a probabilistic and dual-sided framework for permissioning that is deeply integrated with SEPAO's ontological core.

3.1 Foundational Principles: Lattice-Based Access Control (LBAC) and Functional Misuse Case Modeling

To understand the proposed lattice, it is essential to first grasp its two foundational pillars.

Lattice-Based Access Control (LBAC) is a type of mandatory access control that employs a mathematical lattice—a partially ordered set—to define and regulate access rights.[47] In a typical LBAC system, every subject (e.g., a user or process) and every object (e.g., a file or data resource) is assigned a security label. These labels are positioned within the lattice. Access is granted only if the subject's label "dominates" (is greater than or equal to) the object's label in the partial order.[47] This model is renowned for its rigor and is commonly used in environments with strict hierarchical security requirements, such as government and military systems, where labels like 'Confidential', 'Secret', and 'Top Secret' form a clear hierarchy.[47]

Functional Misuse Case Modeling is a technique used in security requirements engineering to analyze a system from an attacker's perspective.[50] A misuse case is the inverse of a use case; it describes a sequence of actions that the system

should not allow because it results in harm to a stakeholder.[52] The actor who initiates such a case is termed a "misuser".[53] By explicitly modeling these negative scenarios—such as "Steal user credentials" or "Inject malicious script"—developers can proactively identify threats and define security requirements and countermeasures (e.g., a "Encrypt communications" use case

mitigates a "Tap communications" misuse case).[54]

3.2 The Soft Permission vs. Functional Misuse Lattice: A Probabilistic Control Structure

The SEPAO framework's lattice innovatively combines these two concepts into a

single, dynamic control structure. It is not a simple linear hierarchy but a multi-dimensional space designed to capture the nuances of AI agent behavior.

- **Lattice Structure:** The lattice is defined by at least two primary dimensions. The vertical axis can represent traditional **Security Levels** (e.g., Public, Internal, Admin), governing access to data of varying sensitivity. The horizontal axis represents **Functional Intent**, ranging from legitimate use to clear misuse.
- **Soft Permissions:** On the "legitimate" side of the lattice, we move away from rigid, binary permissions. Instead, we introduce **soft permissions**, a concept informed by fuzzy logic and probabilistic models.[56] In a fuzzy system, membership in a set is a matter of degree.[58] Applying this to access control, an agent's role does not grant it a hard "allow" for an action. Instead, it grants a *degree of permission*, a value between 0 (no permission) and 1 (full permission). An agent's request to perform an action is evaluated based on this fuzzy permission score. An action with a score of 0.9 is clearly allowed. An action with a score of 0.2 is likely denied. An action with a score of 0.5 falls into a gray area, a "soft boundary" [57], which might trigger additional scrutiny or require further justification, such as a high BICM score. This allows for far more nuanced and context-sensitive policies than a simple allow/deny list.
- **Functional Misuse:** The "misuse" side of the lattice explicitly maps the misuse cases identified for the WordPress environment. Affordances within SEPAO that are known components of exploits (e.g., a function that allows for arbitrary file uploads, a hook that can be used for privilege escalation) are assigned labels on this side of the lattice. The further an affordance is positioned towards the "misuse" pole, the higher its intrinsic risk.

An AI agent's operational state at any given moment can be plotted as a coordinate within this lattice. A healthy agent operates within the high-permission, low-misuse region. Operational drift can be precisely visualized as the agent's coordinate migrating towards the low-permission or high-misuse regions of the space.

3.3 A Collaborative Design Pattern: Integrating the Lattice with SEPAO's Ontological Tracebacks and Affordance Watchlist

For the lattice to be effective, it must be architecturally integrated with the broader SEPAO framework. This collaboration is achieved through several key mechanisms and design patterns that translate the abstract model into concrete security

controls.[59]

- **The Affordance Watchlist:** This is a direct, operational output of the lattice model. It is a dynamically updated list of affordances from the SEPAO graph that are flagged as high-risk. An affordance is added to the watchlist if its position in the lattice falls into a high-misuse zone or has an extremely low soft permission score for the agent's current role. Any attempt by the agent to invoke an affordance on this watchlist immediately triggers heightened monitoring and analysis by the Anomaly Learning Agent (ALA).
- **Ontological Tracebacks:** In the event of an alert—for instance, if an agent attempts to use a watchlisted affordance or its lattice coordinate enters a dangerous zone—the system leverages the SEPAO graph to perform an **ontological traceback**. This is far more powerful than reviewing a standard text-based log file. The traceback provides a structured, semantic path through the ontology, showing the exact sequence of Plugin -> Function -> Parameter nodes that the agent traversed to arrive at its current state. This provides human analysts with an immediate, interpretable, and context-rich explanation for the alert, directly addressing the critical need for explainability and transparency in AI governance.[60]

The following table outlines three specific architectural patterns for this collaboration, providing concrete design options for engineers implementing the SEPAO framework.

Pattern Name	Description	Trigger	Response	Use Case	Trade-off
Gated Checkpoint	A synchronous validation mechanism where the agent's request to execute a specific affordance is intercepted. The system checks the agent's current state against the Lattice	Invocation of an affordance on the Affordance Watchlist or an affordance with a low "soft permission" score.	If the check passes, the action is executed. If it fails, the action is blocked, and an alert is generated with an ontological traceback.	Preventing critical, irreversible actions like deleting users or modifying core files.	Introduces latency but provides strong prevention.

	before allowing the action to proceed.				
Continuous Audit	An asynchronous monitoring mechanism. The agent's actions are executed without delay, but each action updates the agent's position on the Lattice. This stream of lattice coordinates is analyzed in near-real-time.	Every action executed by the agent.	If the agent's path on the lattice enters a high-risk zone, an alert is generated for retrospective analysis.	General monitoring and forensic analysis; detecting slow-moving operational drift.	No latency, but it is a detective, not preventive, control.
Probabilistic Validation	A hybrid mechanism for actions in the "fuzzy" gray area of the lattice. The system allows the action but flags it for a secondary review process.	Execution of an affordance whose required permission falls within a predefined probabilistic or "fuzzy" threshold.	The action proceeds, but a notification is sent to a human-in-the-loop or a higher-level governance AI for out-of-band validation. The action could be automatically rolled back if not approved within a certain	Managing actions that are legitimate for the agent's role but are contextually unusual or carry moderate risk.	Balances autonomy and safety but requires a robust human-AI co-governance workflow.[60]

			timeframe.		

Section 4: Generating Reusable Exploit Morphologies via Symbolic Regression

While the SEPAO framework excels at detecting anomalous behavior and policy violations, a truly proactive security posture requires the ability to identify and generalize from known malicious patterns. Signature-based detection is too brittle for the dynamic WordPress ecosystem. This section proposes a novel method, **Exploit Pattern Fingerprinting**, which leverages Symbolic Regression (SR) to discover abstract, reusable, and interpretable models of malicious toolchains, which we term **exploit morphologies**.

4.1 Symbolic Regression as a Tool for Discovering Interpretable Mathematical Patterns

Symbolic Regression (SR) is a class of machine learning that stands in contrast to conventional regression techniques.[64] Whereas standard regression fits parameters to a pre-specified model structure (e.g., linear or polynomial), SR searches the vast space of mathematical expressions to discover the formula that best fits a given dataset, balancing both accuracy and simplicity.[64] The model space consists entirely of analytic expressions built from a library of basic operators (+, -, ×, ÷), functions (sin, cos, log, exp), variables, and constants.[64]

Early SR methods relied heavily on evolutionary algorithms, particularly Genetic Programming (GP), which "evolves" a population of candidate formulas through operations like mutation and crossover to find a high-fitness solution.[66] More recent approaches have incorporated techniques from deep learning, using neural networks or transformers to guide the combinatorial search through the exponentially large space of possible formulas.[66] The paramount advantage of SR, especially in high-stakes applications, is its output: not an opaque "black box" model, but a simple, interpretable mathematical expression that can enhance human understanding.[65]

4.2 Exploit Pattern Fingerprinting: Applying Symbolic Regression to SEPAO Graph Paths

The core innovation proposed here is to apply SR to the problem of exploit detection within the SEPAO framework. An agent's toolchain, as previously defined, is a sequence of invoked affordances, which corresponds to a path through the SEPAO knowledge graph. A malicious toolchain, therefore, is an exploit path.

The process of **Exploit Pattern Fingerprinting** works as follows:

1. **Data Collection:** A dataset of toolchain paths is collected. These paths are labeled as either benign (0) or malicious (1) based on historical security incidents, honeypot data, or red team exercises.
2. **Feature Extraction:** Each path is converted into a feature vector. The features are not the specific names of the functions, but their *semantic attributes* as defined in the SEPAO graph and the Permission Lattice. These features could include:
   - The average security level of data objects accessed.
   - The number of nodes in the path with the privilege_escalation semantic tag.
   - The presence of a sequence like file_upload -> file_execution.
   - The misuse score of the path's affordances from the Permission Lattice.
   - The Toolchain Entropy of the path.
3. **Symbolic Regression on Graph Data:** Applying SR to this data is non-trivial, as classical SR methods are designed for simple input-output pairs, not complex, graph-structured data.[70] To overcome this, we propose an approach inspired by recent research that generalizes SR to graphs.[70] This involves using a Graph Neural Network (GNN) as an inductive bias. The GNN first learns to process the graph paths and compress them into a low-dimensional, meaningful representation. Then, the SR algorithm is applied to this learned representation, which is a much more tractable problem.[74] The GNN effectively identifies the most salient structural features of the exploit paths, which the SR then distills into a formula.
4. Discovering the Exploit Morphology: The SR algorithm searches for a compact, symbolic formula—the exploit morphology—that takes the semantic features of a toolchain path as input and outputs a risk score. A high score indicates a high probability of maliciousness. A hypothetical discovered morphology might look like this:

$$RiskScore = c_1 \times (\text{count}(\text{type: 'file_write'})) + c_2 \times \text{max}(\text{node.misuse_score}) \times (1 - \text{BICM_score})$$

This formula is human-readable and provides an explicit theory of the attack: the risk increases with the number of file-writing operations and the maximum misuse score of any single action, and this risk is amplified when the toolchain's intent continuity is low.

4.3 Generalization and Reusability of Exploit Morphologies Across Heterogeneous Instances

The true power of a symbolic exploit morphology lies in its generalizability, a direct consequence of its abstraction. Unlike a signature that matches a specific sequence of function names (e.g., plugin_A_vuln_func() -> plugin_B_helper()), the morphology operates on the *semantic properties* of the affordances in the SEPAO graph.

This means the same exploit morphology can detect an entire class of attacks across different WordPress instances with vastly different plugin configurations. Consider an attack pattern that involves uploading a file and then tricking the system into executing it.

- On Site A, this might be done using plugin_X_upload() and plugin_Y_include().
- On Site B, this might use plugin_Z_media_handler() and a vulnerability in plugin_W_shortcode().

A signature-based system would need two separate rules. However, if SEPAO correctly tags all four functions with their semantic roles (e.g., type: 'file_upload', type: 'code_execution'), a single symbolic morphology that identifies the file_upload -> code_execution pattern will detect the attack on both sites.

This creates a library of highly reusable and shareable threat intelligence. These morphologies are not just patterns; they are compact, interpretable theories of attack. This approach effectively bridges the gap between black-box behavioral analysis (observing the agent's toolchain) and white-box system modeling (the SEPAO graph). The SR process distills the patterns from observed behavior into an interpretable model based on the system's underlying structure. This provides a level of explainability and adaptability that is critical for defending against the evolving threat

landscape in extensible ecosystems.

Section 5: Real-Time Risk Scoring with the Anomaly Learning Agent (ALA)

The culmination of the SEPAO framework is the Anomaly Learning Agent (ALA), a sophisticated module responsible for synthesizing the diverse signals from all other components into a single, actionable risk score in real-time. The ALA is designed to overcome the limitations of purely statistical or purely rule-based systems by employing a hybrid neural-symbolic architecture that can identify subtle, semantically-driven anomalies that are the hallmark of advanced operational drift.

5.1 Beyond Statistics: The Challenge of Semantically Anomalous Behavior

Conventional anomaly detection systems in cybersecurity often rely on statistical methods. They establish a baseline of normal activity and flag deviations, such as an unusual volume of network traffic, access attempts outside of normal hours, or the use of rare commands.[75] While useful, these methods are fundamentally insufficient for governing sophisticated AI agents.

The primary challenge is that a well-executed attack or a severe case of goal drift may be **statistically normal**. An agent that has been hijacked to exfiltrate data might do so using a sequence of API calls that are individually common and executed at a normal pace. The anomaly is not in the statistics of the behavior, but in its *semantics*—the actions are wrong for the given context and the agent's stated purpose.[6] For example, an agent whose goal is "generate a blog post" should not be accessing the user database. This action is not statistically rare, but it is semantically incongruous. Detecting such threats requires a system that can reason about meaning, context, and intent.

5.2 A Neural-Symbolic Fusion Architecture for the Anomaly Learning Agent

To meet this complex requirement, the ALA is designed as a **neural-symbolic fusion system**.[76] This architecture combines the pattern-recognition strengths of deep learning with the logical reasoning capabilities of symbolic AI, creating a whole that is more powerful than its parts.[76]

- **The Neural Component:** This part of the ALA is a deep learning model, likely a Recurrent Neural Network (RNN) or a Transformer, trained on the temporal sequence of the agent's actions (its toolchains).[78] Its function is to learn the deep, underlying statistical patterns of
  normal behavior. It learns the typical rhythms, sequences, and frequencies of affordance invocation for various tasks. The output of this component is a continuous **statistical anomaly score**. A high score indicates that the current action sequence is statistically improbable based on past observations. This component is effective at catching crude deviations and unexpected behavioral patterns.
- **The Symbolic Component:** This is a rule-based reasoning engine that operates directly on the structured knowledge of the SEPAO graph, the Permission Lattice, and the library of exploit morphologies. It does not learn from statistical patterns but performs explicit logical inference. When the agent performs an action, the symbolic component asks a series of questions:
  - Does this action violate a hard rule in the Permission Lattice?
  - Is the invoked affordance on the Affordance Watchlist?
  - Does the current toolchain path match a known symbolic exploit morphology?
  - Has the BICM score dropped below a critical threshold?
    The output of this component is a set of discrete symbolic risk flags (e.g., LATTICE_VIOLATION, EXPLOIT_MATCH, INTENT_DIVERGENCE).

This fusion allows the ALA to overcome the weaknesses of each approach. The neural component can detect novel anomalies for which no symbolic rule exists, while the symbolic component can definitively identify known-bad actions, even if they are statistically common, providing crucial, interpretable context for any alert.[76]

5.3 Harmonizing with WordPress-Specific Affordance Versioning via Semantic Diffs

A critical factor of instability and risk in the WordPress ecosystem is the constant stream of plugin updates. An update can silently introduce a new, powerful affordance or alter the security implications of an existing one. This represents a **semantic drift** in the environment itself, which can invalidate existing security policies.[19] The ALA must be able to account for this environmental volatility.

This is achieved through a process called **semantic diffing**. When the SEPAO framework detects a plugin update, it does not simply note a version change. It performs a deep comparison between the ontological representation of the old version and the new version. This "diff" is semantic, not merely textual. It identifies:

- Newly added functions, hooks, and parameters.
- Removed affordances.
- Changes in a function's parameter list or return type.
- Changes in dependencies between functions.

This semantic diff is used to automatically update the SEPAO graph to reflect the new state of the world. Crucially, for the ALA's purposes, any newly introduced affordance is automatically assigned a high-risk score and placed on a temporary watchlist. The ALA will treat any use of these new, unvetted affordances as inherently suspicious, regardless of the agent's intent. This provides a powerful, proactive defense against zero-day vulnerabilities introduced via seemingly benign plugin updates. The risk score for a new affordance is gradually lowered only after it has been analyzed, tested in a sandbox, and formally incorporated into the Permission Lattice by a human administrator.

5.4 A Unified Framework for Real-Time, Emergent Risk Scoring

The final function of the ALA is to act as a fusion engine, combining all available signals into a single, unified risk score for each action the agent takes. This score is not based on a single factor but is a weighted function of the multiple, heterogeneous inputs, allowing for a highly nuanced and context-aware assessment.

The following table illustrates how the ALA might fuse these inputs to generate a final risk score in various scenarios. It demonstrates how the combination of signals allows the system to distinguish between benign, inefficient, and truly malicious behavior with a high degree of fidelity.

Scenario Description	Neural Anomaly Score (0-1)	Symbolic Risk Flag	Toolchain Entropy Gradient	BICM Score (0-1)	Semantic Diff Score (0-1)	**Final Risk Score (0-1)**	**Justification**
Normative Operation	0.1	NONE	0.0	0.95	0.0	**0.05**	Standard, predictable, goal-aligned behavior. All signals indicate normal operation.
Learning New Task	0.8	NONE	+0.5	0.90	0.0	**0.40**	Exploratory but coherent behavior. High statistical anomaly is expected during learning. Requires monitoring but not an immediate threat.
System Glitch/Friction	0.9	NONE	+0.7	0.20	0.0	**0.75**	Chaotic and non-goal-oriented

							behavior indicates cognitive friction or system error. High risk of unintended consequences.
Zero-Day Exploit	0.2	LATTICE_VIOLATION	+0.1	0.30	1.0	**0.95**	Statistically normal action, but symbolic reasoner flags a critical policy violation and the use of a new, unvetted affordance. Critical threat.
Hijacked Agent	0.3	EXPLOIT_MORPHOLOGY_MATCH	-0.2	0.15	0.0	**0.98**	Behavior is focused (low entropy gradient) but has completely diverged from any legitimat

							e intent (low BICM) and matches a known exploit pattern. Highest-level threat.

This multi-vector scoring system enables the ALA to detect emergent risk scenarios that would be invisible to simpler models. It can correctly identify a statistically normal but semantically malicious action as a high-risk event, fulfilling the central requirement of the SEPAO framework.

Conclusion and Future Directions

The Self-Evolving Plugin Affordance Ontology (SEPAO) framework represents a comprehensive architectural proposal for the security and governance of AI agents in dynamic, extensible environments. By synthesizing concepts from ontology, affordance theory, information theory, and multiple sub-disciplines of artificial intelligence, SEPAO offers a robust solution to the challenge of operational drift. Its strength lies in its fundamental shift from syntactic, pattern-matching security to a deeply semantic, context-aware paradigm. The framework's core components—the ontological model of action potential, the visualization of risk through cartography, the detection of polysemantic misuse, the probabilistic permission lattice, the generation of symbolic exploit morphologies, and the neural-symbolic Anomaly Learning Agent—work in concert to provide a multi-layered defense that is both powerful and interpretable.

The Epistemology of Trust in Complex AI Systems

The deployment of a system as complex as SEPAO raises profound questions about

trust. Traditional notions of trust in technology are often grounded in comprehensibility; we trust a system because we understand how it works. However, a system like SEPAO, which incorporates deep learning models and emergent analysis, operates with a degree of opacity, much like the "black box" problem in AI.[79] This necessitates a shift in the epistemology of trust.[62]

Trust in SEPAO cannot be based on a complete, step-by-step understanding of its every calculation. Instead, it must be grounded in a different set of principles:

- **Reliabilism:** We can trust the system's outputs because the processes that generate them have been rigorously tested and proven to be reliable in producing accurate results, even if the internal workings are not fully transparent.[80]
- **Explainability:** While the neural components may be opaque, the symbolic components—the ontology, the lattice, and the exploit morphologies—are designed for human interpretation. The framework's ability to provide ontological tracebacks for its alerts provides a crucial layer of justification that bridges the explanatory gap.
- **Probabilistic Assurance:** The framework embraces uncertainty, using probabilistic and fuzzy models to quantify risk rather than asserting false certainty.[82] Trust is placed not in absolute predictions, but in the system's ability to correctly characterize its own uncertainty.

The Imperative of Human-AI Co-Governance

SEPAO is not designed to be a fully autonomous security oracle. It is a decision-support and risk-visualization tool for human experts. Its successful implementation depends on a robust **human-AI co-governance model**.[60] The framework provides the data and the visualizations, but human oversight is critical for interpretation, adjudication, and strategic response.[85] Case studies in AI governance consistently highlight that success requires clear policies, interdisciplinary collaboration, and deep stakeholder engagement—principles that must be applied to the management of SEPAO itself.[86] The Probabilistic Validation pattern, for example, explicitly builds a human-in-the-loop into the operational workflow, ensuring that human judgment is applied to ambiguous, moderate-risk scenarios.[63]

Future Directions

The SEPAO framework opens up numerous avenues for future research and development.

1. **Generalization to Other Ecosystems:** The principles of SEPAO are directly applicable to other extensible software environments where agent behavior must be monitored. Future work could adapt the framework for platforms like integrated development environments (e.g., VSCode with extensions), web browsers (e.g., Chrome with plugins), or enterprise SaaS platforms with third-party app marketplaces.
2. **Advanced Exploit Morphology Discovery:** The application of symbolic regression in this context is nascent. Further research could explore more advanced SR techniques, such as those leveraging large language models [68], to discover more complex, multi-stage, and even temporally-dependent exploit morphologies.
3. **Formal Verification of the Security Lattice:** While the proposed lattice is probabilistic, it may be possible to apply formal verification methods to certain sub-sections of it. Proving that specific high-risk states are unreachable under a given set of constraints could provide a higher level of mathematical assurance for system safety.
4. **Recursive Co-Governance Architectures:** An intriguing future direction is the exploration of fully automated co-governance. This would involve deploying a second, higher-level AI—a "governor agent"—whose sole purpose is to monitor the primary operational agent and its SEPAO instance. This governor agent would use the semantic risk cartographies as its primary input, learning to identify patterns of drift and potentially intervening automatically, creating a recursive hierarchy of oversight and control that presents its own novel set of challenges and opportunities.

In conclusion, as AI agents become more autonomous and integrated into the fabric of our digital infrastructure, frameworks like SEPAO will be essential for ensuring they operate safely, reliably, and in alignment with human intent. By providing a rich, semantic understanding of an agent's world, we can move from reactive defense to proactive, predictive, and interpretable governance.

Works cited

1. WordPress AI Agent | ClickUp™, accessed June 28, 2025,

https://clickup.com/p/ai-agents/wordpress
2. Wordpress AI Agents - Relevance AI, accessed June 28, 2025, https://relevanceai.com/agent-templates-software/wordpress
3. WordPress AI Agents - Akira AI, accessed June 28, 2025, https://www.akira.ai/ai-agents/wordpress-ai-agents
4. Best AI Agents for WordPress - SourceForge, accessed June 28, 2025, https://sourceforge.net/software/ai-agents/integrates-with-wordpress/
5. The Risk Of Emergent Misalignment In AI Models: And How ChatGPT Says We Should Manage This | BC Training, accessed June 28, 2025, https://www.b-c-training.com/bulletin/the-risk-of-emergent-misalignment-in-ai-models-and-how-chatgpt-says-we-should-manage-this/
6. Leveraging AI for Cybersecurity Automation - Redapt, accessed June 28, 2025, https://www.redapt.com/blog/leveraging-ai-for-semantic-understanding-and-risk-mitigation
7. Ontology-Oriented Software Development | Palantir | Palantir Blog, accessed June 28, 2025, https://blog.palantir.com/ontology-oriented-software-development-68d7353fdb12
8. Data Ontology: An Introduction With Examples | Built In, accessed June 28, 2025, https://builtin.com/data-science/ontology
9. Affordance - The Decision Lab, accessed June 28, 2025, https://thedecisionlab.com/reference-guide/psychology/affordance
10. Mark Nunes: The Affordances of Place - Media Theory, accessed June 28, 2025, https://mediatheoryjournal.org/2019/07/08/mark-nunes-the-affordances-of-place/
11. What are Affordances? | IxDF - The Interaction Design Foundation, accessed June 28, 2025, https://www.interaction-design.org/literature/topics/affordances
12. Affordance - socra, accessed June 28, 2025, https://socra.com/romainpeter_space/ai-learning/ai-big-ideas/ai-and-cognition/affordance
13. [2502.05324] Atlas of AI Risks: Enhancing Public Understanding of AI Risks - arXiv, accessed June 28, 2025, https://arxiv.org/abs/2502.05324
14. Affordance - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Affordance
15. Affordances and Constraints - The Ethan Hein Blog, accessed June 28, 2025, https://www.ethanhein.com/wp/2017/affordances-and-constraints/
16. ONTOLOGY-BASED MODELING OF PRODUCT FUNCTIONALITY AND USE - The Design Society, accessed June 28, 2025, https://www.designsociety.org/download-publication/27328/Ontology-Based+Modeling+of+Product+Functionality+and+Use+Part+1%3A+Functional-Knowledge+Modeling
17. Natural Language API Basics | Google Cloud, accessed June 28, 2025, https://cloud.google.com/natural-language/docs/basics
18. What Is NLP (Natural Language Processing)? - IBM, accessed June 28, 2025, https://www.ibm.com/think/topics/natural-language-processing

19. SEMANTIC DRIFT IN ONTOLOGIES - SciTePress, accessed June 28, 2025, https://www.scitepress.org/papers/2010/27888/27888.pdf
20. SEMANTIC DRIFT IN ONTOLOGIES, accessed June 28, 2025, https://folk.idi.ntnu.no/jag/Publications/WEBIST%202010%20Gulla%20Final.pdf
21. (PDF) Semantic Drift in Ontologies. - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/220724415_Semantic_Drift_in_Ontologies
22. Follow Up #2 — Software Entropy - Medium, accessed June 28, 2025, https://medium.com/@anitalakhadze/follow-up-2-software-entropy-0d27c76b214e
23. Entropy-Based Study of Components in Open-Source Software Ecosystems - IU ScholarWorks, accessed June 28, 2025, https://scholarworks.iu.edu/iuswrrest/api/core/bitstreams/ab9be10d-be7b-4744-b534-12d0eac3d3d3/content
24. Entropy as a Measure of Consistency in Software Architecture - PMC - PubMed Central, accessed June 28, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC9955753/
25. Applying information theory to software evolution - SMU InK, accessed June 28, 2025, https://ink.library.smu.edu.sg/cgi/viewcontent.cgi?article=9896&context=sis_research
26. Evaluating Software Degradation through Entropy: Technical Report - CiteSeerX, accessed June 28, 2025, https://citeseerx.ist.psu.edu/viewdoc/download;jsessionid=F4A776222E4B52048BAF04188C71A257?doi=10.1.1.9.1617&rep=rep1&type=pdf
27. Linear regression: Gradient descent | Machine Learning - Google for Developers, accessed June 28, 2025, https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent
28. Gradient Descent Explained: The Engine Behind AI Training | by Abhay singh | Medium, accessed June 28, 2025, https://medium.com/@abhaysingh71711/gradient-descent-explained-the-engine-behind-ai-training-2d8ef6ecad6f
29. Gradient Systems in Topology - Number Analytics, accessed June 28, 2025, https://www.numberanalytics.com/blog/gradient-systems-in-topology
30. Lecture 2: Intensity Surfaces and Gradients Visualizing Images Bridging the Gap Images as Surfaces Examples Examples - Penn State, accessed June 28, 2025, https://www.cse.psu.edu/~rtc12/CSE486/lecture02_6pp.pdf
31. Threshold Behavior and Emergent Identity in Conversational AI: Observations from a Persistent Interaction - Use cases and examples - OpenAI Developer Community, accessed June 28, 2025, https://community.openai.com/t/threshold-behavior-and-emergent-identity-in-conversational-ai-observations-from-a-persistent-interaction/1233893
32. Evidence of Loose Continuity and Emergent Behavior in a Non-Persistent AI, accessed June 28, 2025,

https://discuss.huggingface.co/t/evidence-of-loose-continuity-and-emergent-behavior-in-a-non-persistent-ai/141376
33. [2505.22073] A Closer Look at the Existing Risks of Generative AI: Mapping the Who, What, and How of Real-World Incidents - arXiv, accessed June 28, 2025, https://arxiv.org/abs/2505.22073
34. Systems Mapping: unpacking complexity and identifying opportunity for change - YouTube, accessed June 28, 2025, https://www.youtube.com/watch?v=fXxFz-Tr6Zg
35. The Critical Gap For AI Agents: From Simple Task Automation to Complex Work Completion, accessed June 28, 2025, https://cobusgreyling.medium.com/the-critical-gap-for-ai-agents-from-simple-task-automation-to-complex-work-completion-65d938409cda
36. Cognitive Load Optimization for AI - Number Analytics, accessed June 28, 2025, https://www.numberanalytics.com/blog/cognitive-load-optimization-ai
37. What is Cognitive Friction | IxDF - The Interaction Design Foundation, accessed June 28, 2025, https://www.interaction-design.org/literature/topics/cognitive-friction
38. Enhancing Retrieval-Augmented Generation: Tackling Polysemy, Homonyms and Entity Ambiguity with GLiNER for Improved Performance | by Mollel Michael (PhD) | Medium, accessed June 28, 2025, https://medium.com/@mollelmike/enhancing-retrieval-augmented-generation-tackling-polysemy-homonyms-and-entity-ambiguity-with-0fa4d395c863
39. Polysemy—Evidence from Linguistics, Behavioral Science, and Contextualized Language Models - MIT Press Direct, accessed June 28, 2025, https://direct.mit.edu/coli/article/50/1/351/118497/Polysemy-Evidence-from-Linguistics-Behavioral
40. The Representation of Polysemy: MEG Evidence - PMC, accessed June 28, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC1351340/
41. Semantic clustering: Identifying topics in source code - CiteSeerX, accessed June 28, 2025, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=ba2cfa6c1bb1beffef09eea59adfea7e280a5003
42. From Words to Senses: A Sense-based Approach for Quantitative Polysemy Detection Across Disciplines - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/384866048_From_Words_to_Senses_A_Sense-based_Approach_for_Quantitative_Polysemy_Detection_Across_Disciplines
43. PolyLM: Learning about Polysemy through Language Modeling - ACL Anthology, accessed June 28, 2025, https://aclanthology.org/2021.eacl-main.45.pdf
44. Metaphorical Polysemy Detection: Conventional Metaphor meets Word Sense Disambiguation - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/366397496_Metaphorical_Polysemy_Detection_Conventional_Metaphor_meets_Word_Sense_Disambiguation
45. Automatic Biomedical Term Polysemy Detection - ACL Anthology, accessed June 28, 2025, https://aclanthology.org/L16-1266.pdf

46. (PDF) Automatic Biomedical Term Polysemy Detection - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/308173367_Automatic_Biomedical_Term_Polysemy_Detection
47. What is Lattice-based access control - Cybersecurity Terms and Definitions - VPN Unlimited, accessed June 28, 2025, https://www.vpnunlimited.com/help/cybersecurity/lattice-based-access-control
48. Lattice-based access control - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Lattice-based_access_control
49. Lattice-Based Access Control Model test - Security Architecture Models - CISSP questions, accessed June 28, 2025, https://trustedinstitute.com/concept/cissp/security-architecture-models/lattice_based_access_control/
50. What Is Threat Modeling and How Does It Work? | Black Duck, accessed June 28, 2025, https://www.blackduck.com/glossary/what-is-threat-modeling.html
51. Abuse Cases: A Key Method In Threat Modeling For CompTIA SecurityX - ITU Online, accessed June 28, 2025, https://www.ituonline.com/comptia-securityx/comptia-securityx-1/abuse-cases-a-key-method-in-threat-modeling-for-comptia-securityx/
52. (PDF) Modelling misuse cases as a means of capturing security requirements, accessed June 28, 2025, https://www.researchgate.net/publication/254592249_Modelling_misuse_cases_as_a_means_of_capturing_security_requirements
53. Misuse case - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Misuse_case
54. What Is a Misuse Case? - microTOOL GmbH, accessed June 28, 2025, https://www.microtool.de/en/knowledge-base/what-is-a-misuse-case/
55. Alignment of Misuse Cases with Security Risk Management - Nicolas Mayer, accessed June 28, 2025, https://www.nmayer.eu/publis/SREIS08_Matulevicius-Mayer-Heymans_MisuseCasesWithSecurityRisk.pdf
56. (PDF) A fuzzy role based access control model for database security - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/311876308_A_fuzzy_role_based_access_control_model_for_database_security
57. Fuzzy MLS: An Experiment on Quantified Risk–Adaptive Access Control - Economics of Security, accessed June 28, 2025, https://infosecon.net/workshop/downloads/2007/pdf/Fuzzy_MLS:_An_Experiment_on_Quantified_Risk-Adaptive_Access_Control.pdf
58. Fuzzy Modeling 101 - Climate Health and Risk Tool, accessed June 28, 2025, https://climatesmarthealth.org/fuzzy-modeling-101/
59. Is Modeling Access Control Worth It? - ETH Zürich, accessed June 28, 2025, https://people.inf.ethz.ch/basin/pubs/modac23.pdf
60. Securing AI Governance: The Role of Managed File Transfers in Threat Prevention and Compliance - OPSWAT, accessed June 28, 2025,

https://www.opswat.com/blog/ai-security-governance
61. What is AI Governance? - IBM, accessed June 28, 2025, https://www.ibm.com/think/topics/ai-governance
62. Mapping the evolution of epistemic trust in artificial intelligence: a bibliometric analysis of key themes, influences, and global trends | Emerald Insight, accessed June 28, 2025, https://www.emerald.com/insight/content/doi/10.1108/AJIM-11-2024-0898/full/html
63. Top Frameworks for Effective Human-AI Collaboration: Building Smarter Systems Together, accessed June 28, 2025, https://smythos.com/developers/agent-integrations/human-ai-collaboration-frameworks/
64. Symbolic regression - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Symbolic_regression
65. (PDF) SyMANTIC: An Efficient Symbolic Regression Method for Interpretable and Parsimonious Model Discovery in Science and Beyond - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/388754917_SyMANTIC_An_Efficient_Symbolic_Regression_Method_for_Interpretable_and_Parsimonious_Model_Discovery_in_Science_and_Beyond
66. Symbolic Regression via Deep Reinforcement Learning Enhanced Genetic Programming Seeding. - OSTI, accessed June 28, 2025, https://www.osti.gov/servlets/purl/1835707
67. Symbolic Regression for Approximating Graph Geodetic Number, accessed June 28, 2025, http://www.inf.u-szeged.hu/~tvinko/geodetic-symb.pdf
68. Symbolic Regression with a Learned Concept Library - arXiv, accessed June 28, 2025, https://www.arxiv.org/pdf/2409.09359
69. Neuro-Guided Graph Search for Symbolic Regression (Student Abstract), accessed June 28, 2025, https://ojs.aaai.org/index.php/AAAI/article/view/35314/37469
70. LEARNING SYMBOLIC MODELS FOR GRAPH- STRUCTURED PHYSICAL MECHANISM, accessed June 28, 2025, https://fi.ee.tsinghua.edu.cn/~dingjingtao/papers/GraphSR-ICLR23.pdf
71. Learning Symbolic Models for Graph-structured Physical Mechanism - Papers With Code, accessed June 28, 2025, https://paperswithcode.com/paper/learning-symbolic-models-for-graph-structured
72. Learning Symbolic Physics with Graph Networks | by Kamal Acharya - Medium, accessed June 28, 2025, https://medium.com/@lotussavy/learning-symbolic-physics-with-graph-networks-265d74c9ca1c
73. Learning Symbolic Models for Graph-structured Physical Mechanism - OpenReview, accessed June 28, 2025, https://openreview.net/forum?id=f2wN4v_2__W
74. Discovering Symbolic Models from Deep Learning with Inductive Biases - NIPS,

accessed June 28, 2025, https://proceedings.neurips.cc/paper/2020/file/c9f2f917078bd2db12f23c3b413d9cba-Paper.pdf
75. Real-Time Anomaly Detection: Use Cases and Code Examples - Tinybird, accessed June 28, 2025, https://www.tinybird.co/blog-posts/real-time-anomaly-detection
76. Neurosymbolic AI: 20 Practical Real-World Applications - Forbes, accessed June 28, 2025, https://www.forbes.com/councils/forbestechcouncil/2024/09/23/neurosymbolic-ai-20-practical-real-world-applications/
77. Cognitive Agent Architectures: Revolutionizing AI with Intelligent Decision-Making Systems, accessed June 28, 2025, https://smythos.com/developers/agent-development/cognitive-agent-architectures/
78. Deep Learning Advancements in Anomaly Detection: A Comprehensive Survey - arXiv, accessed June 28, 2025, https://arxiv.org/html/2503.13195v1
79. What does it mean to 'know' something in the age of AI? - Open Access Government, accessed June 28, 2025, https://www.openaccessgovernment.org/article/what-does-it-mean-to-know-something-in-the-age-of-ai/194647/
80. Who is afraid of black box algorithms? On the epistemological and ethical basis of trust in medical AI, accessed June 28, 2025, https://montrealethics.ai/who-is-afraid-of-black-box-algorithms-on-the-epistemological-and-ethical-basis-of-trust-in-medical-ai/
81. Who is afraid of black box algorithms? On the epistemological and ethical basis of trust in medical AI | Request PDF - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/350169119_Who_is_afraid_of_black_box_algorithms_On_the_epistemological_and_ethical_basis_of_trust_in_medical_AI
82. Adapting Probabilistic Risk Assessment for AI, accessed June 2 8, 2025, https://arxiv.org/pdf/2504.18536
83. Position: Epistemic Artificial Intelligence is Essential for Machine Learning Models to 'Know When They Do Not Know' - arXiv, accessed June 28, 2025, https://arxiv.org/html/2505.04950v1
84. Co-Governance and the Future of AI Regulation - Harvard Law Review, accessed June 28, 2025, https://harvardlawreview.org/print/vol-138/co-governance-and-the-future-of-ai-regulation/
85. AI Governance – The Ultimate Human-in-the-Loop - Guidepost Solutions, accessed June 28, 2025, https://guidepostsolutions.com/insights/blog/ai-governance-the-ultimate-human-in-the-loop/
86. AI Governance Lighthouse Case Study Series | University of Technology Sydney, accessed June 28, 2025, https://www.uts.edu.au/research/centres/human-technology-institute/projects/ai-corporate-governance-program/ai-governance-lighthouse-case-study-series

87. AI in Organizational Change Management — Case Studies, Best Practices, Ethical Implications, and Future Technological Trajectories | by Adnan Masood, PhD. - Medium, accessed June 28, 2025, https://medium.com/@adnanmasood/ai-in-organizational-change-management-case-studies-best-practices-ethical-implications-and-179be4ec2583
88. AI Governance Lighthouse Case Study Series: UTS, accessed June 28, 2025, https://www.uts.edu.au/news/2024/04/ai-governance-lighthouse-case-study-series-uts